Technical Report NETCFG-TR-001

NetCfg: A Declarative Network Configuration Compiler

Technical Reference

Simon Knight

March 2026

Abstract. NetCfg is a Rust-based network configuration compiler that transforms high-level topology intent into verified, target-specific device configurations. The compiler processes intent through a five-layer pipeline: (1) Intent Transformation via composable primitives, (2) Topological Validation via a DSL-driven assertion engine, (3) Lowering to a vendor-neutral Device Intermediate Representation (DeviceIR), (4) Target-Specific AST Transformation (TSDM) for hardware-aware lowering, and (5) Syntax Serialization via dumb templates. This report documents the data model, the transformation DSLs, and the AST-based lowering mechanism that decouples architectural logic from physical hardware constraints.

Version	Date	Changes
1.0	March 2026	Initial release (v1.3 feature set)
1.1	March 2026	Updated to v2.0: policy primitives, WASM plugins, LSP, REST API, 4 new CLI commands

Contents

1 Introduction

What NetCfg is

NetCfg is a command-line tool and Rust library that compiles declarative network topology descriptions into per-device configuration files. It takes a *blueprint* (describing what the network should look like) and a *mapping* (describing how to extract vendor-specific configuration from the topology) and produces deterministic, diffable, auditable configuration artifacts for every device in the network.

NetCfg replaces the manual "glue" layer between source-of-truth systems (NetBox, Nautobot), automation frameworks (Ansible, Nornir), and network modelling tools (Batfish). It treats network configuration as a *compilation problem*: topology intent is compiled through a typed pipeline, not interpolated through ad-hoc templates.

Design philosophy

Two principles explain every design decision in this document:

1. **Lowering over templating.** The operator's intent is lowered through successive stages of abstraction. It moves from a high-level graph to a neutral AST (DeviceIR), then to a platform-specific AST (TSDM), and only finally to CLI syntax. This ensures that hardware quirks and vendor-specific logic are handled via structured AST transformations, not ad-hoc string concatenation.
2. **Separation of what from how.** The blueprint declares *what* the network should look like (topology transformations). The mapping declares *how* to extract device configuration from the result. The same blueprint can produce Cisco IOS, Arista EOS, or Juniper JunOS output by swapping only the mapping and templates.

Audience

This document is intended for:

- Engineers integrating NetCfg into a network automation pipeline.
- Contributors to the NetCfg and NTE codebases.
- Researchers evaluating graph-based approaches to network configuration.

Familiarity with YAML, basic graph theory, and IP networking is assumed. Experience with Rust is helpful but not required for the API sections.

How to read this document

Readers wanting to get started quickly should read §2 (data model), §3 (blueprint DSL), and §6 (CLI reference), then refer to individual sections as needed. §4–§5 cover the compilation pipeline and core mechanisms in depth and can be deferred until required. §7 provides a complete end-to-end worked example that traces a blueprint through every pipeline stage. §9 documents the language server and REST API for editor and CI/CD integration.

2 Background: Network Configuration as Code

NetCfg models the network as a directed graph $G = (V, E, \tau, \lambda, D)$ where V is a set of nodes (devices, protocol instances), $E \subseteq V \times V$ is a set of edges (links, peering sessions),

$\tau : V \to T$ assigns each node a *type* (Router, Switch), $\lambda : V \to L$ assigns each node a *layer*, and $D : T \to \mathit{DataFrame}$ maps each type to a columnar property store (Polars DataFrame).

2.1 Layers

A layer represents a plane of the network: `physical` for the hardware topology, `bgp` for BGP peering sessions, `ospf` for OSPF adjacencies. A single physical router (node 1, layer `physical`) may have corresponding protocol nodes (node 101, layer `bgp`; node 201, layer `ospf`), each linked to node 1 via a *parent mapping* (`_source_id` in the node's `data_json`).

Layers allow the compiler to model the physical and logical topologies simultaneously, with cross-layer references preserving traceability from a protocol node back to the physical device that hosts it.

2.2 Per-node state: DeviceContext

Each node's properties are stored as a JSON blob (`data_json`) in a Polars DataFrame keyed by node type. NetCfg deserialises this blob into a typed `DeviceContext` struct:

Listing 1: DeviceContext: the per-node state model.

```
pub struct DeviceContext {
    pub hostname: String,
    pub device_os: Option<String>,        // template selector
    pub management_ip: Option<String>,
    pub src_ip: Option<String>,           // written by provision_ips
    pub dst_ip: Option<String>,           // written by provision_ips
    pub subnet: Option<String>,           // written by provision_ips
    pub src_ipv6: Option<String>,         // dual-stack support
    pub dst_ipv6: Option<String>,
    pub subnet_v6: Option<String>,
    pub vrf: Option<String>,              // VRF domain
    pub peer_interfaces: Option<HashMap<String, String>>,
    pub next_interface_index: Option<u32>,
    pub stanzas: Vec<Stanza>,             // accumulated config units
    pub metadata: HashMap<String, JsonValue>, // extension point
}
```

Primitives progressively enrich this context: `mesh_nodes` establishes edges and writes `mesh_type`/`peer_count` into metadata; `provision_ips` writes IP fields; `build_protocol_layer` deep-merges config overlays into the cloned node's metadata. By the time rendering occurs, each node's `DeviceContext` contains all information needed to produce its configuration file.

Note

The `metadata` field uses `serde(flatten)`, so any JSON key not matching a named field is captured here. This makes `DeviceContext` extensible without schema changes.

2.3 Graph engine dependency

The underlying graph is implemented by the NTE topology engine, which provides a petgraph-backed directed graph with Polars DataFrames for columnar property storage. NetCfg depends on `nte_topology` for the graph and `ank_nte` for the mapping evaluator and DeviceIR types.

3 Blueprint DSL: Declarative Topology Transformations

A blueprint is a YAML document declaring what topology transformations to apply. It is the "what" of the network design.

3.1 Schema and layer ordering

Listing 2: Blueprint top-level schema.

```
pub struct Blueprint {
    pub version: u32,              // must be >= 1
    pub imports: Vec<String>,      // relative paths to merge
    pub layers: Vec<LayerSpec>,    // ordered list of layer blocks
    pub assertions: Vec<AssertionSpec>,  // post-execution checks
}

pub struct LayerSpec {
    pub name: String,              // non-empty
    pub requires: Vec<String>,     // must name earlier layers
    pub primitives: Vec<Primitive>,
}
```

Three validation passes run at parse time:

1. **Schema validation** via `serde_valid`: field types, non-empty names, integer ranges. Unknown YAML keys are rejected via `deny_unknown_fields`, catching typos at parse time.
2. **CEL selector compilation**: each selector string is compiled into an evaluable CEL program, surfacing syntax errors before execution.
3. **Layer ordering validation**: a forward scan ensures every `requires` entry names a previously-declared layer. This catches ordering bugs at parse time.

The optional `imports` field lists relative file paths to other blueprint files. At parse time, layers and assertions from imported files are merged into the importing blueprint, enabling reusable assertion libraries (e.g. a standard linting ruleset shared across projects).

Warning

Duplicate layer names are permitted. Multiple `LayerSpec` blocks with `name: input` are common in practice—each block adds primitives to the same layer. The `requires` mechanism enforces inter-layer ordering, not intra-layer uniqueness.

3.2 Selector DSL (CEL surface syntax)

Selectors identify which nodes or edges a primitive operates on. The surface syntax is a lightweight wrapper around Google's Common Expression Language (CEL):

Listing 3: Selector syntax examples.

```
# Select all nodes
selector: "all()"

# Select nodes by property
selector: "nodes[site='a']"
selector: "nodes[role=='spine' && layer=='input']"
selector: "nodes[type=='Router' and site='dc1']"

# Select edges
selector: "edges[src>=0]"
selector: "edges[true]"
```

The selector parser:

1. Extracts the target (`nodes` or `edges`) and the filter expression from the bracket syntax.
2. Normalises operators: single `=` becomes `==`; `and`/`or`/`not` become `&&`/`||`/`!`; single `&`/`|` become doubled.

3. Compiles the normalised expression as a CEL program.

At evaluation time, the selector iterates over all nodes (or edges) in the topology, binds each entity's properties as CEL variables (`id`, `type`, `layer`, plus all `data_json` fields), and evaluates the CEL program. Entities for which the program returns `true` are included in the match set.

Note

Undeclared variable references (e.g., querying a field that some nodes lack) silently evaluate to `false` rather than producing an error. This allows heterogeneous topologies where not all nodes have the same properties.

For IP-valued fields, the selector also binds a `{field}_len` variable containing the prefix length, enabling selectors like `nodes[subnet_len == 30]`.

3.3 Primitive catalogue

NetCfg provides twelve primitive types. All primitives follow the *Selector* → *Execute* → *Mutate* pattern and the *two-pass borrow* discipline: collect all reads into local vectors (immutable borrows), then apply all mutations (mutable borrows), satisfying Rust's ownership rules without `unsafe` code.

Table 1 summarises the catalogue. Full field tables follow.

Table 1: Primitive catalogue overview.

Primitive	Purpose
`mesh_nodes`	Create edges between selected nodes (full, hub-and-spoke, route-reflector)
`provision_ips`	Allocate IP subnets to edges from a CIDR pool
`build_protocol_layer`	Clone nodes into a named protocol overlay layer
`allocate_resources`	Generic pool allocation for ASNs, VLANs, etc.
`global_pool`	Register a named IP pool for hierarchical carving
`global_resource_pool`	Register a named resource pool (non-IP)
`conditional`	CEL-based branching within blueprints
`custom_primitive`	Named reusable primitive group with parameters
`inject_secrets`	Read environment variables into node metadata
`build_routing_policy`	Attach routing policy stanzas (route-maps) to matched nodes
`build_access_policy`	Attach access-control policy stanzas (ACLs) to matched nodes
`wasm_plugin`	Execute a WebAssembly plugin binary against matched nodes

3.3.1 `mesh_nodes`

Creates edges between selected nodes. Supports three mesh topologies: `full` ($\binom{n}{2}$ edges), `hub_and_spoke` (hubs full-meshed, spokes connected to all hubs), and `route_reflector` (same topology as hub-and-spoke). Idempotent: existing edges are detected via a `HashSet<(i32, i32)>` and skipped.

Table 2: `mesh_nodes` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for nodes to mesh
`mesh_type`	String	Yes	`full`, `hub_and_spoke`, or `route_reflector`
`hub_selector`	String	H&S	Selector for hub nodes
`spoke_selector`	String	H&S	Selector for spoke nodes
`edge_properties`	JSON	No	Properties for created edges (deferred; see §10.1)
`naming_strategy`	String	No	Interface naming: `eth`, `cisco_ge`, `junos_ge`

Example: full mesh within a site

```
- type: mesh_nodes
  selector: "nodes[site='a']"
  mesh_type: full
  naming_strategy: eth
```

With 4 nodes at site A, this creates $\binom{4}{2} = 6$ edges and assigns interface names `eth0`, `eth1`, etc. to each node's `peer_interfaces` map.

3.3.2 `provision_ips`

Allocates subnets from a CIDR pool to edges. See §5.1 for the full five-pass algorithm.

Table 3: `provision_ips` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for edges
`pool`	String	Yes	CIDR (e.g. `10.0.0.0/24`) or global pool name
`subnet_size`	u8	No	Prefix length per edge (default: 30 for IPv4, 126 for IPv6)
`ipv6_pool`	String	No	IPv6 CIDR or global pool name for dual-stack
`ipv6_subnet_size`	u8	No	IPv6 prefix length per edge
`strategy`	String	No	`dense` (default), `sparse`, or `hash`
`pool_size`	u32	No	When referencing a global pool, size of block to carve
`vrf`	String	No	VRF domain (default: `default`)

3.3.3 `build_protocol_layer`

Clones physical nodes into a named protocol layer. See §5.2 for the overlay construction mechanism.

Table 4: `build_protocol_layer` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for source-layer nodes
`layer`	String	Yes	Target layer name (must differ from source)
`config`	JSON	Yes	Config overlay deep-merged into cloned nodes
`clone_underlying`	bool	No	Also clone edges between selected nodes (default: false)

3.3.4 `allocate_resources`

Generic pool allocator for non-IP resources (ASNs, VLANs, loopback IDs).

Table 5: `allocate_resources` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for nodes
`resource_type`	String	Yes	Key name in `DeviceContext.metadata`
`pool`	String	Yes	Range (e.g. `65000-65999`) or global pool name
`strategy`	String	No	`dense` (default)
`pool_size`	u32	No	When referencing a global pool, number of values to carve

3.3.5 `global_pool`

Registers a named IP address pool at blueprint scope, enabling hierarchical pool carving across multiple `provision_ips` primitives.

Table 6: `global_pool` fields.

Field	Type	Required	Description
`name`	String	Yes	Pool identifier referenced by `provision_ips`
`pool`	String	Yes	CIDR range (e.g. `10.0.0.0/8`)
`vrf`	String	No	VRF domain (default: `default`)

3.3.6 `global_resource_pool`

Registers a named non-IP resource pool at blueprint scope.

Table 7: `global_resource_pool` fields.

Field	Type	Required	Description
`name`	String	Yes	Pool identifier
`resource_type`	String	Yes	Resource category (e.g. `asn`)
`pool`	String	Yes	Value range (e.g. `65000-65999`)

3.3.7 `conditional`

CEL-based branching. The `condition` is evaluated as a CEL expression; if true, `then_primitives` execute, otherwise `else_primitives` (if present) execute.

Table 8: `conditional` fields.

Field	Type	Required	Description
`condition`	String	Yes	CEL expression
`then_primitives`	Vec<Primitive>	Yes	Primitives if condition is true
`else_primitives`	Vec<Primitive>	No	Primitives if condition is false

3.3.8 `custom_primitive`

A named, reusable group of primitives with parameters. The `parameters` map is pushed onto a stack and accessible within the nested primitives.

Table 9: `custom_primitive` fields.

Field	Type	Required	Description
`name`	String	Yes	Primitive name (for reporting)
`parameters`	HashMap<String, JsonValue>	Yes	Named parameters
`primitives`	Vec<Primitive>	Yes	Nested primitive list

3.3.9 `inject_secrets`

Reads environment variables into node metadata. Keys in the `secrets` map are metadata field names; values are environment variable names.

Table 10: `inject_secrets` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`secrets`	HashMap<String, String>	Yes	{metadata_key: env_var_name}

3.3.10 `build_routing_policy`

Attaches routing policy stanzas to matched nodes. Each statement defines a route-map entry with match conditions and set actions, stored as a `Stanza` in `DeviceContext.stanzas` for downstream template rendering.

Table 11: `build_routing_policy` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`name`	String	Yes	Policy name (e.g. `BGP-EXPORT`)
`statements`	Vec<RoutingPolicyStatement>	Yes	Ordered route-map entries

Each `RoutingPolicyStatement` has fields: `name` (sequence number), `action` (`permit` or `deny`), optional `match_condition`, and optional `set_action`. Templates access stanzas via `device.stanzas | selectattr('kind', 'equalto', 'routing_policy')`.

Example: BGP export policy

```
- type: build_routing_policy
  selector: "nodes[role='border']"
  name: "BGP-EXPORT"
  statements:
    - name: "10"
      action: permit
      match_condition: "prefix-list␣CUSTOMER"
      set_action: "local-preference␣100"
    - name: "20"
      action: deny
```

3.3.11 `build_access_policy`

Attaches access-control policy stanzas to matched nodes. Each rule defines a firewall or ACL entry with source, destination, protocol, and port fields.

Table 12: `build_access_policy` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`name`	String	Yes	Policy name (e.g. `EDGE-ACL`)
`rules`	Vec<AccessPolicyRule>	Yes	Ordered ACL entries

Each `AccessPolicyRule` has fields: `name`, `action` (`permit`/`deny`), optional `source`, `destination`, `protocol`, and `port`. Both policy primitives enforce `SelectorTarget::Nodes`—policies cannot target edges.

Example: DMZ access control list

```
- type: build_access_policy
  selector: "nodes[zone='dmz']"
  name: "UNTRUSTED-TO-INTERNAL"
  rules:
    - name: "100"
      action: deny
      protocol: tcp
      destination: "10.0.0.0/8"
    - name: "999"
      action: permit
```

3.3.12 `wasm_plugin`

Executes a WebAssembly plugin binary against matched nodes. Requires the `wasm` Cargo feature flag (`--features wasm`); without it, the pipeline returns a descriptive error. Uses the `wasmtime` v18 runtime.

Table 13: `wasm_plugin` fields.

Field	Type	Required	Description
`selector`	String	Yes	CEL selector for target nodes
`plugin_path`	String	Yes	Filesystem path to compiled `.wasm` binary

The execution pipeline: (1) load and compile the `.wasm` module, (2) instantiate per matched node without host imports, (3) resolve the `apply_node` exported symbol, (4) pass the node's `DeviceContext` as serialised JSON. The current implementation validates compilation and symbol resolution; full linear memory I/O for JSON exchange is planned for a future release (see §11).

Note

The `wasm` feature is optional to avoid the `wasmtime` dependency (which adds ∼40 MB to the binary) for users who do not need plugin extensibility.

3.4 Design-rule assertions

Blueprints may declare assertions: post-execution invariants checked after all primitives complete but before commit. Each assertion has a severity: `error` (pipeline failure) or `warning` (diagnostic only).

Listing 4: Design-rule assertions in a blueprint.

```
assertions:
  - name: "all␣routers␣have␣hostname"
    severity: error
    select: all()
    check:
      field_exists: hostname
  - name: "IPs␣within␣allocation"
    severity: error
    select: "nodes[src_ip␣!=␣null]"
    check:
      field_in_cidr:
        field: src_ip
        cidr: "10.0.0.0/8"
  - name: "unique␣hostnames␣per␣site"
    severity: warning
    select: all()
    check:
      unique_per_group:
        field: hostname
        group_by: site
  - name: "private␣ASN␣range"
    severity: warning
    select: "nodes[asn␣!=␣null]"
    check:
      custom_cel:
        expression: "asn␣>=␣64512␣&&␣asn␣<=␣65534"
```

Seven check types are available:

Table 14: Assertion check types.

Check	Semantics
`field_exists`	Every matched node has the named field in its `data_json`
`min_count`	The matched set contains at least N items
`max_count`	The matched set contains at most N items
`min_edges`	Every matched node has at least N outgoing edges
`unique_per_group`	Within groups defined by `group_by`, the `field` value is unique
`field_in_cidr`	Every matched node's IP field is contained within the specified CIDR
`custom_cel`	Arbitrary CEL expression evaluated per matched entity; fails if false

This enables "policy as code" directly in the blueprint. Assertions are evaluated cross-layer (no layer filter), so a single assertion can validate properties across both physical and protocol layers.

4 Compilation Pipeline

NetCfg's compilation pipeline has five distinct layers of abstraction, operating as a true network compiler. Figure 1 shows the data flow from abstract intent to concrete syntax.

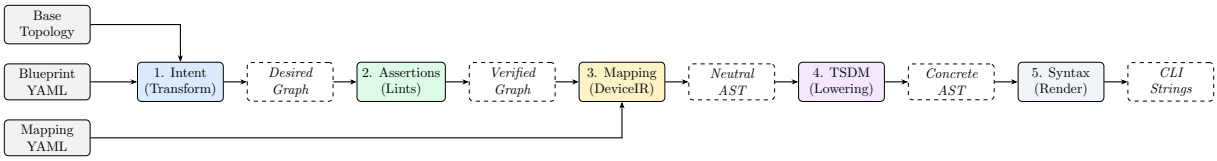

Figure 1: NetCfg compilation pipeline. High-level intent is progressively lowered through three distinct AST representations (Verified Graph, Neutral AST, Concrete AST) before serialization.

4.1 Layer 1: Intent Transformation

The blueprint is parsed into an AST and applied to a transactional shadow copy of the topology. Composable primitives (IPAM, Meshing, Overlays) mutate the graph to reach the desired state.

4.2 Layer 2: Topological Assertions

A DSL-driven assertion engine validates the desired graph against architectural invariants (e.g. area IDs must match, MTU consistency). Compilation aborts if invariants are violated.

4.3 Layer 3: DeviceIR Mapping

A Mapping DSL lowers the rich topology graph into a vendor-neutral **Abstract Syntax Tree (DeviceIR)**. This representation captures logical intent (e.g. "Neighbor X is remote-as Y") without hardware context.

4.4 Layer 4: Target-Specific Transform (TSDM)

The **TSDM Layer** performs AST-to-AST transformations. It lowers the Neutral AST into a platform-specific AST, remapping logical interfaces to physical ports (e.g. `Eth1` → `Eth1/1/1`) based on the target's chassis model and hardware layout.

4.5 Layer 5: Syntax Serialization

Jinja2 templates act as a dumb emission layer, serializing the final Target-Specific AST into vendor-specific CLI syntax. Zero logic is implemented at this layer.

The `TransactionalOutput` writer collects all files in temporary locations (using `NamedTempFile` in the same directory for same-filesystem guarantee), then atomically renames all via POSIX `rename(2)` on commit. If any stage fails, all temp files are dropped without committing—no partial output directory is ever visible.

Three artifacts are emitted per node:

- {`hostname`}`.conf`: the rendered device configuration.
- {`hostname`}`.deviceir.json`: the raw DeviceIR stanzas (audit).
- {`hostname`}`.lineage.json`: per-line mapping from config lines to the `rule_id` that produced them (provenance tracing).

4.5.1 Determinism guarantee

Given a fixed blueprint B, mapping M, and input topology G, the output configuration set $C = \mathit{compile}(G, B, M)$ is deterministic. Serialisation round-trip produces a bit-identical shadow. Primitives execute in declaration order. Selectors evaluate deterministically. Node ID allocation is monotonic. IP allocation iterates edges sorted by $(\mathit{src}, \mathit{dst})$. DeviceIR stanzas use `BTreeMap` and are post-sorted. Template rendering is pure.

5 Core Mechanisms

5.1 Five-pass IPAM (`provision_ips`)

The IP allocator (`primitives/ipam.rs`) implements a five-pass algorithm:

1. **State discovery**: Scan all nodes for existing IP assignments (`src_ip`, `dst_ip`, `subnet` fields).
2. **Edge selection**: Evaluate the CEL selector; sort matched edges by (src, dst) for determinism.
3. **Pre-reservation**: Insert existing allocations into the allocator, preventing re-allocation and enforcing stickiness.
4. **Fulfilment**: For each edge, allocate a subnet using the configured strategy (`dense`, `sparse`, or `hash`). Extract host addresses from the subnet.
5. **Write-back**: Write `src_ip`, `dst_ip`, `subnet` (and IPv6 equivalents for dual-stack) into each endpoint's `DeviceContext`.

Identity-based stickiness. The allocator uses a canonical identity —`sorted(hostname_a, hostname_b)`—as the primary key. If a prior allocation exists for the same identity, the same prefix is returned without pool search. This anchors allocations to stable semantic identity, not integer IDs. If topology IDs change (e.g., after a re-import) but hostnames remain the same, allocations are preserved.

VRF isolation. The allocator maintains per-VRF state. Two primitives using the same CIDR pool in different VRFs can allocate overlapping subnets without conflict. Pool overlap *within* the same VRF is detected and rejected.

Allocation strategies:

- `dense`: Sequential allocation from the pool start. First edge gets the first available subnet.
- `sparse`: Skips every other subnet, leaving growth headroom between allocations.
- `hash`: Subnet position is derived from a hash of the edge identity, producing topology-independent allocations that do not depend on evaluation order.

Hierarchical pools. A `global_pool` primitive registers a large CIDR (e.g., `10.0.0.0/8`). Subsequent `provision_ips` primitives reference the pool by name and specify a `pool_size` (e.g., 24), carving /24 blocks from the parent pool on demand.

Dual-stack. When `ipv6_pool` is specified, both IPv4 and IPv6 addresses are allocated in a single primitive invocation. IPv6 fields (`src_ipv6`, `dst_ipv6`, `subnet_v6`) are written alongside IPv4 fields.

5.2 Protocol overlay construction (`build_protocol_layer`)

`build_protocol_layer` clones physical nodes into a named protocol layer. For each selected source-layer node:

1. Allocate a new monotonic node ID.
2. Add the node to the target layer.
3. Deep-merge the config overlay into the cloned node's metadata.
4. Record `_source_id` as a parent mapping to the physical node.
5. Optionally (`clone_underlying: true`) clone edges between source-layer nodes into the overlay.

Cross-layer traceability: the `_source_id` field links each protocol node to its physical origin. Since `provision_ips` already enriched the physical node's `DeviceContext` with IP addresses, the cloned protocol node inherits them via deep merge—this is how IP addresses flow from IPAM allocation to BGP configuration without explicit wiring.

Deep merge semantics: When both the base node and the config overlay contain nested JSON objects under the same key, the merge recurses key-by-key (overlay wins on conflict). This preserves sibling keys that the overlay does not mention. For example, a base node with {`bgp: {as: 65000, holdtime: 180}`} and an overlay with {`bgp: {timers: {keepalive: 60}}`} produces {`bgp: {as: 65000, holdtime: 180, timers: {keepalive: 60}}`}.

Idempotency: The primitive scans the target layer for existing nodes with `_source_id` matching source nodes. Already-cloned nodes are skipped, so re-running the primitive after a partial failure produces the correct result.

5.3 Configuration diffing (DiffEngine)

The diff engine (`diff/engine.rs`) computes a `MutationPlan` between two topologies by exporting each layer to JSON and comparing:

1. **Node diff**: Per layer, compare node sets by ID. Detect additions, deletions, and modifications (field-by-field property comparison via `extract_properties`, which unpacks the `data_json` blob).
2. **Edge diff**: Compare by (src, dst) pair with structural property comparison.

The `MutationPlan` contains six change types: `AddNode`, `RemoveNode`, `UpdateNode`, `AddEdge`, `RemoveEdge`, `UpdateEdge`. Changes are sorted deterministically by (change_type, layer, id/src/dst) for reproducible output.

The `plan` command displays the plan as a human-readable diff; the `ci-run` command uses it to compute per-device configuration diffs.

5.4 AST Transformations (TSDM Layer)

The **Target-Specific Device Model (TSDM)** layer implements the compiler's lowering phase via DSL-driven **AST-to-AST transformations**. This stage bridges the gap between logical design and physical hardware constraints.

5.4.1 The Transformation DSL

Lowering rules are defined in the blueprint using an expressive rewrite-rule grammar. Each rule consists of a `match_expr` (CEL predicate) and an `apply` map (field mutations).

Example: NX-OS interface lowering

```
transforms:
  - name: "nxos_lowering"
    when: "device_os == 'nxos'"
    rules:
      - match_expr: "kind == 'interface' && name.startsWith('Ethernet')"
        apply:
          name: "name + '/1'"  # Ethernet1 -> Ethernet1/1
          mtu: "9216"          # Force jumbo frames
```

5.4.2 Execution Logic

The transformation engine (`dsl/transform.rs`) performs a deep-walk of the DeviceIR stanzas. For each stanza, it:

1. Binds the stanza's fields as CEL variables.
2. Evaluates the `match_expr` predicate.
3. If matched, executes the `apply` expressions to mutate the stanza's state.

This structured approach allows complex hardware logic—such as mapping logical ports to specific line-card slots in a modular chassis—to be expressed as programmable transformations rather than hardcoded logic or brittle templates.

The key architectural insight is that `DeviceContext` accumulates state across the entire primitive pipeline:

1. `mesh_nodes`: writes `mesh_type`, `peer_count`, and optionally `peer_interfaces` into metadata.
2. `provision_ips`: writes `src_ip`, `dst_ip`, `subnet`, `vrf`.
3. `build_protocol_layer`: deep-merges all of the above into the cloned protocol node, plus the config overlay.
4. `allocate_resources`: writes the allocated resource value (e.g., `asn`) into metadata.

By the time Stage 4 (rendering) occurs, each node's `DeviceContext` carries hostname, IPs, protocol metadata, interface assignments, and any custom fields—the complete information needed to produce a device configuration file.

6 CLI Reference

NetCfg provides eleven CLI commands spanning the development-to-deployment lifecycle. All commands use `clap` for argument parsing.

6.1 `netcfg generate`

Runs the full six-stage pipeline, writing artifacts to disk.

```
netcfg generate <topology> <blueprint> \
    --templates-dir <DIR> \
    [--output-dir <DIR>] [--emit-ir] [--verbose]
```

Argument	Default	Description
`topology`	—	Path to base topology archive (.nte)
`blueprint`	—	Path to blueprint YAML
`--templates-dir`	—	Directory containing `.j2` templates
`--output-dir`	`./output`	Output directory for generated files
`--emit-ir`	false	Also write {`hostname`}`.ir.json`
`--verbose`	false	Print per-node rendering progress

Output: one `.cfg` file per device (plus `.ir.json` if `--emit-ir`). Absolute paths are printed to stdout, one per line, suitable for piping.

6.2 `netcfg validate`

Executes the pipeline in dry-run mode: all primitives run with `dry_run=true`, skipping DataFrame writes but verifying what *would* change. Optionally pre-flights templates by rendering to memory.

```
netcfg validate <topology> <blueprint> \
    [--templates-dir <DIR>] [--verbose]
```

Exit codes: 0 on success, non-zero on validation failure (parse errors, assertion failures, template syntax errors, IP pool conflicts).

6.3 `netcfg plan`

Computes the topology `MutationPlan` (current vs. desired) and displays additions, removals, and property changes. With `--diff`, it renders before/after configs and shows a unified text diff per device using the `similar` crate.

```
netcfg plan <topology> <blueprint> \
    [--diff] [--templates-dir <DIR>] [--verbose]
```

Output without `--diff`: colourised plan (green for additions, red for removals, yellow for modifications). Summary line: `+N additions, -N removals across M layers`.

Output with `--diff`: unified diff per device, preceded by `diff --cfg {hostname}`.

6.4 `netcfg inspect`

Provides three views into intermediate state:

```
netcfg inspect <path> --ast               # Print blueprint AST
netcfg inspect <path> --topology          # Export as Graphviz DOT
    [--blueprint <FILE>]
netcfg inspect <path> --trace <NODE_ID>   # Trace node state changes
    --blueprint <FILE>
```

- `--ast`: Parses the file as a blueprint and prints the AST tree.
- `--topology`: Exports the graph as Graphviz DOT. With `--blueprint`, applies the blueprint first. Output includes per-layer subgraph clustering, hostname labels, and subnet annotations.
- `--trace <node_id>`: Replays primitive execution and prints before/after `DeviceContext` snapshots for the specified node at each primitive step.

6.5 `netcfg ci-run`

The CI/CD integration point. Runs validate, plan, and config diff in a single invocation.

```
netcfg ci-run <topology> <blueprint> \
    --templates-dir <DIR> [--json]
```

With `--json`, emits a structured payload:

```
{
  "status": "success",
  "topology_additions": 42,
  "topology_removals": 0,
  "configuration_diffs": {
    "core-0": "-␣␣ip␣address␣10.0.0.4/30\n+␣␣ip␣address␣10.0.0.8/30"
  },
  "warnings": []
}
```

This integrates with GitHub Actions, GitLab CI, or any CI system that can parse JSON.

6.6 `netcfg import`

Imports a YAML topology definition into an NTE archive.

```
netcfg import <yaml_path> <output.nte>
```

The importer reads a topogen-format YAML file (defining nodes with types, layers, and properties) and serialises it to an `.nte` archive that other commands consume.

6.7 `netcfg lint`

Runs design-rule assertions without the full validation pipeline. Applies the blueprint in dry-run mode, then evaluates assertions and reports results.

```
netcfg lint <topology> <blueprint> [--verbose]
```

Returns assertion results with pass/fail status, severity, and diagnostic messages. Exit code 1 only when `error`-severity assertions fail; warnings are reported but do not cause failure.

6.8 `netcfg fmt`

Standardises blueprint YAML formatting. Parses the blueprint via `Executor::parse()`, re-serialises with `serde_yml`, and compares byte-for-byte. Idempotent.

```
netcfg fmt <blueprint> [--check] [--verbose]
```

With `--check`, exits with code 1 if formatting changes are needed (useful as a pre-commit hook or CI gate). Without `--check`, overwrites the file in place.

6.9 `netcfg impact`

Calculates the operational blast radius of a blueprint change against a base topology.

```
netcfg impact <topology> <blueprint> [--verbose]
```

For each mutation in the computed plan, assigns a risk level:

Risk	Colour	Trigger
High	Red	Node removal; critical property change (`asn`, `router_id`, `src_ip`, `subnet`); physical link add/remove
Medium	Yellow	Node addition; logical link add/remove
Low	Green	Metadata-only update

Critical properties are: `asn`, `router_id`, `src_ip`, `dst_ip`, `subnet`, `loopback`, `mgmt_ip`. Output includes per-node risk attribution with reasons.

6.10 `netcfg report`

Generates an automated Markdown architecture report from a compiled topology.

```
netcfg report <topology> <blueprint> -o <output.md> [--verbose]
```

The report contains three sections:

1. **Node inventory**: hostname, type, OS, ASN, loopback, management IP.
2. **Physical links & IP addressing**: source/destination hosts, interfaces, IPs, subnets.
3. **Topology visualisation**: A `netviz-json` code block with a custom JSON schema (`NetVizNode`, `NetVizEdge`, `NetVizTopology`) suitable for rendering by external visualisation tools.

6.11 `netcfg export-clab`

Generates a Containerlab `clab.yaml` simulation file from a compiled topology, enabling rapid lab deployment for testing (e.g. the three-site mesh from §7).

```
netcfg export-clab <topology> <blueprint> \
    -o <clab.yaml> [--configs_dir <DIR>] [--verbose]
```

Heuristically maps `device_os` to Containerlab container kinds: `cisco_iosxr` → `vr-xrv9k`, `nokia_srl` → `srl`, `arista_eos` → `ceos`, default → `linux`. When `--configs_dir` is provided, bind-mounts generated configuration files into OS-specific remote paths (e.g. `/etc/opt/srlinux/config.json` for SRL, `/mnt/flash/startup-config` for cEOS).

7 End-to-End Worked Example

We trace the `three-site-mesh.yaml` blueprint through every pipeline stage. The base topology has 12 routers across 3 sites (4 per site: site-a-r1 through site-c-r4).

7.1 Input: Blueprint

Listing 5: three-site-mesh.yaml (abridged).

```yaml
version: 1
layers:
  # Stage 1: Full mesh within each site
  - name: input
    primitives:
      - type: mesh_nodes
        selector: "nodes[site='a']"
        mesh_type: full
      - type: mesh_nodes
        selector: "nodes[site='b']"
        mesh_type: full
      - type: mesh_nodes
        selector: "nodes[site='c']"
        mesh_type: full

  # Stage 2: IP allocation per site
  - name: input
    primitives:
      - type: provision_ips
        selector: "edges[src>=0]"
        pool: "10.1.0.0/24"
        subnet_size: 30
        strategy: dense

  # Stage 3: BGP overlay
  - name: input
    primitives:
      - type: build_protocol_layer
        selector: "nodes[true]"
        layer: bgp
        config:
          protocol_type: bgp
          asn_base: "65000"
```

7.2 Stage 1: Parse and validate

The parser validates the YAML, confirms `version: 1`, and compiles each selector string (`nodes[site='a']`, `edges[src>=0]`, etc.) to a CEL program. The normaliser converts `site='a'` to `site=="a"`.

7.3 Stage 2: Shadow topology

The executor clones the base topology via a serialisation round-trip, creating an isolated shadow copy. All subsequent mutations operate on the shadow; the base topology is never modified.

7.4 Stage 3: Primitive execution

mesh_nodes (three invocations): Creates $3 \times \binom{4}{2} = 18$ edges (6 per site). The selector `nodes[site='a']` matches the 4 nodes with {`site: "a"`} in their `data_json`. Each node gets `mesh_type: "full"` and `peer_count: 3` written to its metadata.

provision_ips: The selector `edges[src>=0]` matches all 18 edges. Edges are sorted by (src, dst). The five-pass algorithm allocates /30 subnets from `10.1.0.0/24`:

- Edge (1,2): subnet `10.1.0.0/30`, IPs `10.1.0.1` and `10.1.0.2`
- Edge (1,3): subnet `10.1.0.4/30`, IPs `10.1.0.5` and `10.1.0.6`
- ... continuing through all 18 edges.

After this stage, node `site-a-r1` has: `src_ip: "10.1.0.1", subnet: "10.1.0.0/30", vrf: "default"`.

build_protocol_layer: Clones all 12 physical nodes into the `bgp` layer (new IDs 13–24). Each clone receives:

- `protocol_type: "bgp"` and `asn_base: "65000"` (from the config overlay)
- `_source_id: 1` (parent mapping)
- All properties from the physical node, including the IPs written by `provision_ips` (via deep merge)

7.5 Stage 4: Mapping evaluation

The companion mapping (`three-site-mesh-mapping.yaml`):

```
rules:
  - selector: "all_nodes"
    stanza:
      kind: "bgp"
      fields:
        node: "site-a-r1"
        local_asn: "65001"
        description: "site-a␣full-mesh␣BGP␣session"
```

The evaluator matches all nodes and emits one stanza per match. Each stanza carries `provenance:` {`rule_id: "rule_0"`}.

7.6 Stage 5: Render

Stanzas are grouped by `fields.node` (all mapped to `site-a-r1` in this example). The fallback template produces:

```
kind=bgp key=
```

7.7 Stage 6: Atomic write

Three files are written atomically:

- `site-a-r1.conf`: the rendered configuration
- `site-a-r1.deviceir.json`: the raw stanza list
- `site-a-r1.lineage.json`: per-line provenance

8 Error Model

NetCfg uses typed error enums with `thiserror` derivation and `miette` diagnostic annotations. Table 15 summarises the error types.

Table 15: Error enums and their variants.

Enum	Variants	Context
`ParseError`	`YamlSyntax`, `Validation`, `Deserialize`, `LayerOrdering`	Blueprint parsing
`SelectorError`	`InvalidSyntax`, `Compile`, `Evaluate`, `NonBooleanResult`, `Polars`, `Json`, `TopologyAccess`	Selector compilation and evaluation
`PrimitiveError`	`Validation`, `Execute`, `Selector`, `Topology`, `Graph`, `Polars`, `Json`	Primitive execution
`ExecuteError`	`Parse`, `Primitive`, `Shadow`, `Plan`, `Assertion`	Top-level executor
`ShadowError`	`Clone`, `Commit`, `Validation`	Shadow topology lifecycle
`AssertionError`	`Failed`, `Selector`	Assertion evaluation
`DiffError`	`TopologyAccess`, `ParseError`	Configuration diffing
`RenderError`	`Io`, `Jinja`, `Json`, `MissingTemplate`	Template rendering

`ExecuteError` variants carry `miette::Diagnostic` annotations with diagnostic codes (e.g., `netcfg::parse`, `netcfg::primitive::execution_failed`, `netcfg::assertion`) enabling structured error handling in CI pipelines.

`ParseError::YamlSyntax` includes a source span (`miette::SourceSpan`) pointing at the offending line, enabling IDE-quality error rendering.

9 Editor and Service Integration

9.1 Language Server (LSP)

The `netcfg-lsp` crate provides a Language Server Protocol implementation built on `tower-lsp` and Tokio. It communicates via stdin/stdout and offers three capabilities:

1. **Real-time diagnostics.** On file open, change, and save, the server calls `parse_blueprint()` and maps errors to LSP diagnostics with line-column precision. For `serde_yml` errors (which embed location as free text), a regex extracts "at line N column M". Diagnostics are published with source label `ank_netcfg` and severity `ERROR`.
2. **Primitive auto-completion.** When the cursor follows a `type:` token (detected by a suffix heuristic), the server offers completion items for the six core primitives, each with a one-line docstring. Trigger characters are space and colon.

3. **Document synchronisation.** Open documents are tracked in a `DashMap<String, String>` (lock-free concurrent map). Full-text sync (`TextDocumentSyncKind::FULL`) is used for correctness. On close, the document is removed and diagnostics are cleared.

9.2 REST API Microservice

The `netcfg-api` crate exposes the compiler as a stateless HTTP service via `axum`, enabling CI/CD pipelines and enterprise orchestrators to compile blueprints without a local `netcfg` binary.

Endpoint: `POST /compile` (port 3000).

Table 16: Compile endpoint request and response schema.

Direction	Field	Type	Description
Request	`topology_zip_base64`	String	Base64-encoded `.nte` archive
Request	`blueprint_yaml`	String	Raw YAML blueprint
Response	`status`	String	`success` or `error`
Response	`configs`	Map<String, String>	Hostname → rendered config
Response	`device_ir`	Map<String, JSON>	Hostname → DeviceContext IR
Response	`warnings`	Vec<String>	Compilation warnings

The compilation pipeline decodes the base64 payload into a temporary file, loads the topology, parses the blueprint, executes via `Executor::apply()`, and extracts per-device IR. All processing is in-memory with no persistent state; the service scales horizontally behind a load balancer.

10 Limitations

10.1 `edge_properties` not applied (`mesh_nodes`)

Description. The `MeshNodesSpec.edge_properties` field is accepted by the blueprint parser but intentionally not applied. Applying it requires an edge DataFrame API on `nte_topology::Topology` equivalent to `set_dataframe`/`update_dataframe` but keyed on $(source\_id, target\_id)$ pairs.

Workaround. Edge properties can be encoded as node metadata on both endpoints (e.g., `link_bandwidth` as a per-node field keyed by peer ID).

Planned resolution. Awaiting `nte-topology` $\geq$0.2 edge DataFrame support. A tracking test (`edge_properties_deferred_until_nte_edge_dataframe`) is present in the test suite.

10.2 Mapping AST inspection not implemented

Description. The `inspect --ast` command can parse and display blueprint ASTs but does not yet support mapping documents. Attempting to inspect a mapping file prints "Mapping AST visualization coming soon."

Workaround. Inspect the mapping YAML directly. The format is simple (a `rules` list with `selector` and `stanza` fields).

Planned resolution. When the mapping DSL grows beyond simple selector/stanza pairs, AST visualisation will be added.

10.3 Shadow validation is a placeholder

Description. The `ShadowTopology::validate()` method currently only checks that the shadow did not become empty. It does not perform structural integrity checks (dangling edges, orphaned protocol nodes, etc.).

Workaround. Design-rule assertions can catch many structural problems. Use `min_edges` and `field_exists` assertions as guardrails.

Planned resolution. Integrate with the NTE policy validation framework when available.

10.4 No intra-primitive rollback

Description. If a primitive fails partway through execution (e.g., `provision_ips` exhausts its pool after allocating half the edges), the shadow topology retains the partial mutations. The pipeline aborts (returning an error), and the shadow is discarded, but there is no mechanism to roll back individual primitives within a shadow.

Workaround. This is usually not a problem: the entire shadow is discarded on error, so no partial state reaches production. The limitation matters only for debugging: the error message may not reflect the full extent of partial mutations.

Planned resolution. No current plan. Intra-primitive rollback would require snapshotting the shadow before each primitive, which doubles memory usage.

10.5 `generate` CLI bypasses mapping document

Description. The `generate` CLI command renders configs directly from `DeviceContext` via templates, bypassing the mapping document. The full six-stage pipeline (with mapping evaluation and DeviceIR) is implemented in `config_gen::generate_configs()` but the CLI does not expose it yet.

Workaround. Use the `generate_configs()` function directly from Rust code for the full pipeline with mapping support.

Planned resolution. Add `--mapping` flag to the `generate` CLI command.

10.6 No incremental compilation

Description. Every invocation recompiles the entire topology from scratch. The diff engine can identify which devices changed, but the pipeline cannot skip unchanged devices during compilation.

Workaround. Use `ci-run --json` to compute the diff, then selectively push only the changed configuration files.

Planned resolution. Under investigation. Incremental compilation requires caching intermediate state (the desired topology) between runs and detecting which blueprint changes invalidate which nodes.

11 Future Work

- **GPU-accelerated primitive execution**: Offloading selector evaluation and IPAM allocation to the GPU via NTE's WebGPU compute backend, targeting 100K+ node topologies.

- **WASM linear memory I/O**: The current `wasm_plugin` primitive validates module compilation and symbol resolution but mocks the JSON exchange via linear memory. Full C-ABI memory allocation between the Rust host and WASM guest is required for production use.
- **API template rendering**: The `POST /compile` endpoint currently returns placeholder configuration text; integrating the MiniJinja rendering engine into the API service is planned.
- **Mapping CLI integration**: Exposing the full six-stage pipeline (including mapping evaluation) through the `generate` CLI command.
- **Incremental compilation**: Caching intermediate state to avoid recompiling unchanged portions of the topology.
- **LSP assertion integration**: Extending the language server to evaluate design-rule assertions in real time, providing inline warnings for selector violations and IP pool capacity.

12 References

1. DigitalOcean. NetBox: The premier network source of truth. https://netbox.dev/, 2024.
2. Network to Code. Nautobot: The Network Source of Truth and Automation Platform. https://www.networktocode.com/nautobot/, 2024.
3. Red Hat. Ansible: Simple IT Automation. https://www.ansible.com/, 2024.
4. Nornir contributors. Nornir: A pluggable multi-threaded framework. https://nornir.tech/, 2024.
5. Intentionet. Batfish: Network configuration analysis. https://www.batfish.org/, 2024.
6. Ivan Pepelnjak. netlab: Virtual networking lab framework. https://netlab.tools/, 2024.
7. Google. Common Expression Language. https://cel.dev/, 2024.
8. Simon Knight. NTE: A Hybrid Graph-Relational Engine for Network Topology Management. Technical Report, 2026.

A Blueprint YAML Schema Reference

Listing 6: Complete blueprint schema (annotated).

```
version: 1                    # integer, >= 1
imports: [<path>, ...]        # optional, relative file paths to merge

layers:
  - name: <string>            # non-empty, layer identifier
    requires: [<string>, ...] # optional, previously-declared layers
    primitives:

      # --- mesh_nodes ---
      - type: mesh_nodes
        selector: <cel_selector>
        mesh_type: full | hub_and_spoke | route_reflector
        hub_selector: <cel_selector>      # required for h&s
        spoke_selector: <cel_selector>    # required for h&s
        edge_properties: <json_object>    # deferred
        naming_strategy: eth | cisco_ge | junos_ge

      # --- provision_ips ---
      - type: provision_ips
        selector: <cel_selector>
        pool: <cidr_or_pool_name>
        subnet_size: <u8>                 # default: 30 (v4), 126 (v6)
        ipv6_pool: <cidr_or_pool_name>    # optional dual-stack
        ipv6_subnet_size: <u8>
        strategy: dense | sparse | hash   # default: dense
        pool_size: <u32>                  # for global pool carving
        vrf: <string>                     # default: "default"

      # --- build_protocol_layer ---
      - type: build_protocol_layer
        selector: <cel_selector>
        layer: <string>                   # target layer
        config: <json_object>             # deep-merged into clone
        clone_underlying: <bool>          # default: false

      # --- allocate_resources ---
      - type: allocate_resources
        selector: <cel_selector>
        resource_type: <string>
        pool: <range_or_pool_name>
        strategy: dense                   # default: dense
        pool_size: <u32>

      # --- global_pool ---
      - type: global_pool
        name: <string>
        pool: <cidr>
        vrf: <string>

      # --- global_resource_pool ---
      - type: global_resource_pool
        name: <string>
        resource_type: <string>
        pool: <range>

      # --- conditional ---
      - type: conditional
        condition: <cel_expression>
        then_primitives: [<primitive>, ...]
        else_primitives: [<primitive>, ...]   # optional

      # --- custom_primitive ---
      - type: custom_primitive
        name: <string>
        parameters: {<string>: <json_value>}
        primitives: [<primitive>, ...]
```

```
    # --- inject_secrets ---
    - type: inject_secrets
      selector: <cel_selector>
      secrets: {<metadata_key>: <env_var_name>}

    # --- build_routing_policy ---
    - type: build_routing_policy
      selector: <cel_selector>
      name: <string>                    # policy name
      statements:
        - name: <string>               # sequence number
          action: permit | deny
          match_condition: <string>    # optional
          set_action: <string>         # optional

    # --- build_access_policy ---
    - type: build_access_policy
      selector: <cel_selector>
      name: <string>                    # policy name
      rules:
        - name: <string>
          action: permit | deny
          source: <string>             # optional
          destination: <string>        # optional
          protocol: <string>           # optional
          port: <string>               # optional

    # --- wasm_plugin (requires --features wasm) ---
    - type: wasm_plugin
      selector: <cel_selector>
      plugin_path: <filesystem_path>    # path to .wasm binary

assertions:
  - name: <string>
    severity: error | warning
    select: <cel_selector>
    check:
      field_exists: <field_name>
      # OR
      min_count: <integer>
      # OR
      max_count: <integer>
      # OR
      min_edges: <integer>
      # OR
      unique_per_group:
        field: <field_name>
        group_by: <field_name>
      # OR
      field_in_cidr:
        field: <field_name>
        cidr: <cidr>
      # OR
      custom_cel:
        expression: <cel_expression>
```

B Selector DSL Grammar

Listing 7: Selector DSL grammar (informal EBNF).

```
selector     ::= "all()" | target "[" filter "]"
target       ::= "nodes" | "edges"
filter       ::= cel_expression

(* Surface syntax normalisations applied before CEL compilation *)
"="          -> "=="
```

```
"and"        -> "&&"
"or"         -> "||"
"not"        -> "!"
"&"          -> "&&"
"|"          -> "||"

(* CEL variables bound per entity *)
nodes:  id (int), type (string), layer (string), + all data_json fields
edges:  id (string), src (int), dst (int), layer (string),
        + all data_json fields

(* IP prefix length extraction *)
For any field with an IP/CIDR value "10.0.0.0/30":
  {field}_len = 30  (integer, bound automatically)

(* Undeclared references *)
Accessing a field that does not exist on a node evaluates to false
(not an error), enabling heterogeneous topologies.
```

C CLI Quick Reference

Table 17: CLI command summary.

Command	Synopsis
`generate`	Full pipeline: parse → transform → render → write
`validate`	Dry-run pipeline with optional template pre-flight
`plan`	Compute and display topology mutation plan (or config diff)
`inspect`	AST visualisation, Graphviz DOT export, or per-node trace
`ci-run`	Combined validate + plan + config diff for CI/CD
`import`	Import YAML topology to NTE archive
`lint`	Run design-rule assertions only
`fmt`	Standardise blueprint YAML formatting
`impact`	Calculate blast radius of a blueprint change
`report`	Generate Markdown architecture report
`export-clab`	Generate Containerlab simulation file